# 3.3 数据框



•Pandas中另一个重要的数据对象为数据框（DataFram），由多个序列按照相同的index组织在一起形成一个二维表。事实上，数据框的每一列为序列。数据框的属性包括index、列名和值。由于数据框是更为广泛的一种数据组织形式，许多外部数据文件读取到Python中大部分会采用数据框的形式进行存取，比如数据库、excel和TXT文本。同时数据框也提供了极为丰富的方法用于处理数据及完成计算任务。数据框是Python完成数据处理及分析的最重要数据结构之一，因此学会灵活运用数据框是利用Python进行数据处理及挖掘的关键环节。

---

•下面我们主要介绍数据框的创建、属性、方法和数据的访问及切片等内容。

# 3.3.1 数据框创建



基于字典，利用Pandas库中的DataFrame函数，可以创建数据框。其中字典的键转化为列名，字典的值转化为列值，而索引为默认值，即从0开始从小到大排列。

```
import pandas as pd
import numpy as np
data={'a':[2,2,np.nan,5,6],'b':['kl','kl','kl',np.nan,'kl'],'c':[4,6,
5,np.nan,6],'d':[7,9,np.nan,9,8]}
df=pd.DataFrame(data)
```

如下图

df - DataFrame

| Index | a | b | c | d |
|---|---|---|---|---|
| 0 | 2 | kl | 4 | 7 |
| 1 | 2 | kl | 6 | 9 |
| 2 | nan | kl | 5 | nan |
| 3 | 5 | nan | nan | 9 |
| 4 | 6 | kl | 6 | 8 |

# 3.3.2 数据框属性



•数据框对象具有三个属性，分别为列名、索引和值。比如以上3.3.1定义的数据框df，可以通过以下示例程序获取并打印其属性结果。

```
print('columns= ')
print(df.columns)
print('-'*50)
print('index= ')
print(df.index)
print('-'*50)
print('values= ')
print(df.values)
```

输出结果为：

```
columns=
Index(['a', 'b', 'c', 'd'], dtype='object')
index= RangeIndex(start=0, stop=5, step=1)
--------------------------------------------------
values=
[[2.0 'kl' 4.0 7.0]
 [2.0 'kl' 6.0 9.0]
 [nan 'kl' 5.0 nan]
 [5.0 nan nan 9.0]
 [6.0 'kl' 6.0 8.0]]
```

# 3.3.3 数据框方法



•数据框（DataFrame）作为数据处理及挖掘分析的重要基础数据结构，提供了非常丰富的方法用于数据处理及计算。下面介绍其常用的方法，包括去掉空值（nan值）、对空值（nan值）进行填充、基于字段列值进行排序、基于index进行排序、取前N行数据、删除列、数据框之间的连接、数据框转化为Numpy数组、数据导出到Excel、相关统计分析等。下面分别给出其详细的介绍。

# 3.3.3数据框方法--去掉空值



**1. dropna()**

通过dorpna()方法，可以去掉数据集中的空值（nan值），需要注意的是原来数据集不发生改变，新数据集需要重新定义。以3.3.1定义的数据框df为例，示例代码如下：

df1=df.dropna()

执行结果如图3-8所示。

df1 - DataFrame

| Index | a | b | c | d |
|---|---|---|---|---|
| 0 | 2 | k1 | 4 | 7 |
| 1 | 2 | k1 | 6 | 9 |
| 4 | 6 | k1 | 6 | 8 |

图3-8

## 2. fillna()

通过fillna()方法，可以对数据框中的空值（nan值）进行填充。默认情况下所有空值填充同一个元素值（数值或者字符串），也可以指定不同的列填充不同的值。以3.3.1定义的数据框df为例，示例代码如下：

```
df2=df.fillna(0)        #所有空值元素填充0
df3=df.fillna('Kl')      #所有空值元素填充kl
df4=df.fillna({'a':0,'b':'kl','c':0,'d':0})  #全部列填充
df5=df.fillna({'a':0,'b':'kl'}) #部分列填充
```

执行结果如图3-9所示。

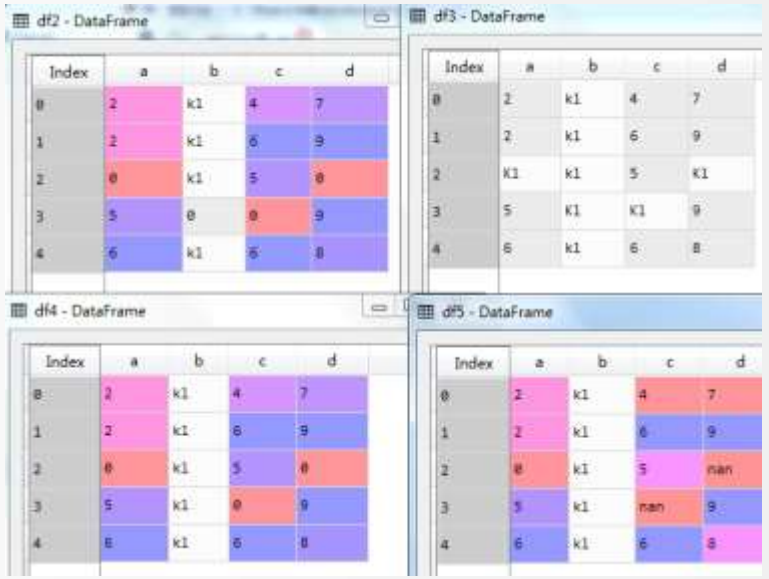



图3-9

**3. sort_values()**

可以利用sort_values()方法，指定列按值进行排序，示例代码如下：

```
import pandas as pd
data={'a':[5,3,4,1,6],'b':['d','c','a','e','q'],'c':[4,6,5,5,6]}
Df=pd.DataFrame(data)
Df1=Df.sort_values('a',ascending=False)
 #默认按升序，这里设置为降序
```

执行结果如图3-10所示。

Df - DataFrame

| Index | a | b | c |
|---|---|---|---|
| 0 | 5 | d | 4 |
| 1 | 3 | c | 6 |
| 2 | 4 | a | 5 |
| 3 | 1 | e | 5 |
| 4 | 6 | q | 6 |

Df1 - DataFrame

| Index | a | b | c |
|---|---|---|---|
| 4 | 6 | q | 6 |
| 0 | 5 | d | 4 |
| 2 | 4 | a | 5 |
| 1 | 3 | c | 6 |
| 3 | 1 | e | 5 |

**4. sort_index()**

有时候需要对索引进行排序，这时候可以使用sort_index()方法，以前面定义的Df1为例，示例代码如下：

Df2=Df1.sort_index(ascending=False) #默认按升序，这里设置为降序

执行结果如图3-11所示。

图3-11

## 5. head()

通过head(N)方法，可以取数据集中的前N行，比如取前面定义的数据框Df2中的前4行，示例代码如下：

H4=Df2.head(4);

执行结果如图3-12所示。

H4 - DataFrame

| Index | a | b | c |
|---|---|---|---|
| 4 | 6 | q | 6 |
| 3 | 1 | e | 5 |
| 2 | 4 | a | 5 |
| 1 | 3 | c | 6 |

图3-12

**6. drop()**

利用dorp()方法，可以删掉数据集中的指定列。比如删除前面定义的H4中的b列，示例代码如下：

```
H41=H4.drop('b',axis=1) #需指定轴为1
```

执行结果如图3-13所示。

H41 - DataFrame

| Index | a | c |
|---|---|---|
| 4 | 6 | 6 |
| 3 | 1 | 5 |
| 2 | 4 | 5 |
| 1 | 3 | 6 |

图3-13

## 7. join()

利用join()方法，可以实现两个数据框之间的水平连接，示例代码如下：

```
Df3=pd.DataFrame({'d':[1,2,3,4,5]})
Df4=Df.join(Df3)
```

执行结果如图3-14所示。

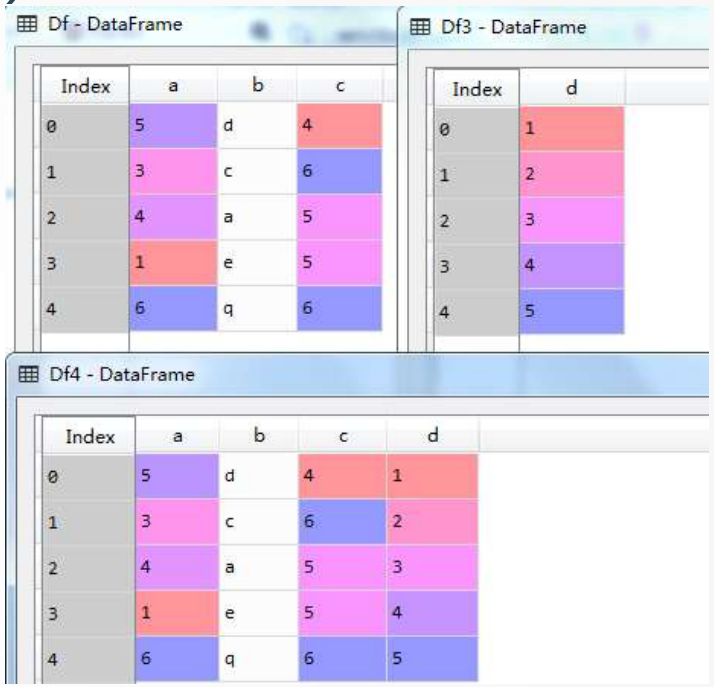



图3-14

## 8. as_matrix()

可以利用as_matrix()方法，将数据框转换为Numpy数组的形式，方便程序使用，特别是数据框中的数据全为数值数据的时候更为有效。示例代码如下：

```
import pandas as pd
list1=['a','b','c','d','e','f']
list2=[1,2,3,4,5,6]
list3=[1.4,3.5,2,6,7,8]
list4=[4,5,6,7,8,9]
list5=['t',5,6,7,'k',9.6]
D={'M1':list1,'M2':list2,'M3':list3,'M4':list4,'M5':list5} #定义字典D，值为字符、数值混合数据
G={'M1':list2,'M2':list3,'M3':list4}                #定义字典G，值为纯数值数据
D=pd.DataFrame(D)                                   #将字典D转化为数据框
D1=D.as_matrix()                                    #将数据框D转化为Numpy数组D1
G=pd.DataFrame(G)                                   #将字典G转化为数据框
G1=G.as_matrix()                                    #将数据框G转化为Numpy数组G1
```

执行结果如图3-15所示。

## 8. as_matrix()

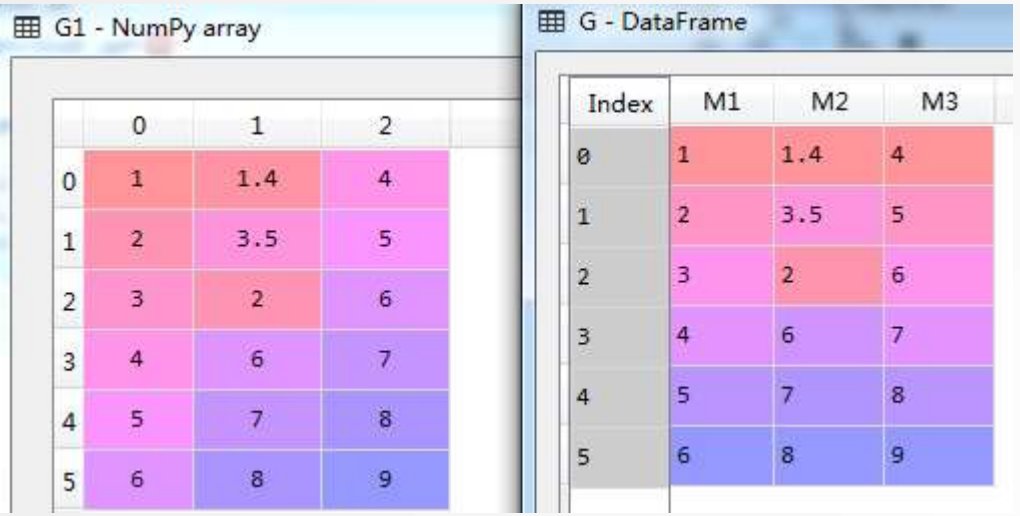



图3-15

而D不是纯的数值数据，转换后的Numpy数组在Spyder变量资源管理器中无法查看，但可以打印出来在控制台窗口中查看，通过print(D1)打印得到如下结果：

```
[['a' 1 1.4 4 't']
 ['b' 2 3.5 5 5]
 ['c' 3 2.0 6 6]
 ['d' 4 6.0 7 7]
 ['e' 5 7.0 8 'k']
 ['f' 6 8.0 9 9.6]]
```

因此，如果数据框中的数据为纯数值的时候，通过转换为Numpy数值数组，使用起来显得更加方便。

## 9. to_excel()

Excel作为常用的数据处理软件，在日常工作中经常用到，通过to_excel()方法，可以将数据框导出到Excel文件中，比如将前面定义的D和G两个数据框导出到Excel文件中。示例代码如下：

```
D.to_excel('D.xlsx')
G.to_excel('G.xlsx')
```

执行结果如图3-16所示

| | M1 | M2 | M3 | M4 | M5 |
|---|---|---|---|---|---|
| 0 | a | 1 | 1.4 | 4 | t |
| 1 | b | 2 | 3.5 | 5 | 5 |
| 2 | c | 3 | 2 | 6 | 6 |
| 3 | d | 4 | 6 | 7 | 7 |
| 4 | e | 5 | 7 | 8 | k |
| 5 | f | 6 | 8 | 9 | 9.6 |

| | M1 | M2 | M3 |
|---|---|---|---|
| 0 | 1 | 1.4 | 4 |
| 1 | 2 | 3.5 | 5 |
| 2 | 3 | 2 | 6 |
| 3 | 4 | 6 | 7 |
| 4 | 5 | 7 | 8 |
| 5 | 6 | 8 | 9 |

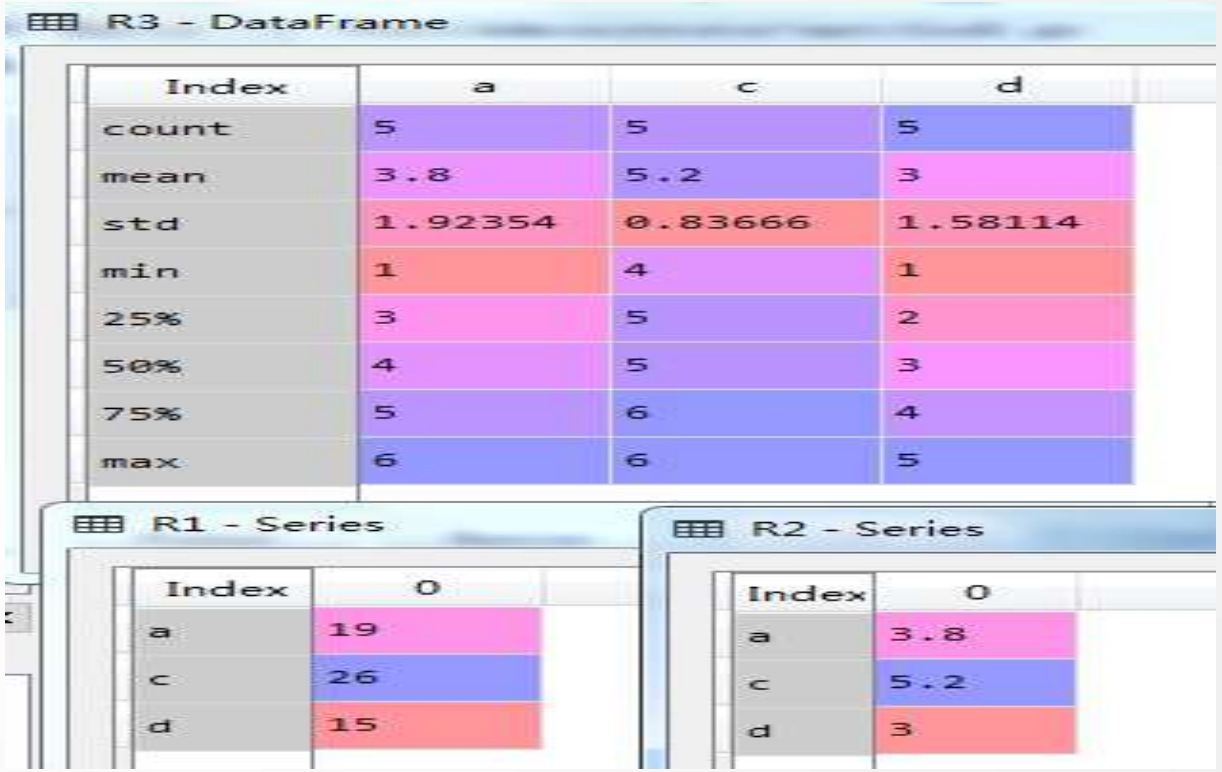
图3-16

## 10. 统计方法

可以对数据框中各列求和、求平均值或者进行描述性统计，以前面定义的Df4为例，示例代码如下：

```
Dt=Df4.drop('b',axis=1)  #Df4中删除b列
R1=Dt.sum()              #各列求和
R2=Dt.mean()             #各列求平均值
R3=Dt.describe()         #各列做描述性统计
```

结果如图3-17所示。



R3 - DataFrame

| Index | a | c | d |
|---|---|---|---|
| count | 5 | 5 | 5 |
| mean | 3.8 | 5.2 | 3 |
| std | 1.92354 | 0.83666 | 1.58114 |
| min | 1 | 4 | 1 |
| 25% | 3 | 5 | 2 |
| 50% | 4 | 5 | 3 |
| 75% | 5 | 6 | 4 |
| max | 6 | 6 | 5 |

R1 - Series

| Index | 0 |
|---|---|
| a | 19 |
| c | 26 |
| d | 15 |

R2 - Series

| Index | 0 |
|---|---|
| a | 3.8 |
| c | 5.2 |
| d | 3 |

图3-17

# 3.3.4数据框切片



**1. 利用数据框中的iloc属性进行切片**

与数组切片类似，利用数据框中的iloc属性可以实现下标值或者逻辑值定位索引，并进行切片操作。假设DF为待访问或切片的数据框，则访问或者切片的数据=DF.iloc[①,②]。其中①为对DF的行下标控制，②为对DF的列下标控制，行和列下标控制通过数值列表来实现，但是需要注意的是列表中的元素不能超出DF中的最大行数和最大列数。为了更灵活地操作数据，取所有的行或者列，可以用“：”来代替实现。同时，行控制还可以通过逻辑列表来实现。以3.3.3中定义的df2为例，示例代码如下：

```
# iloc for positional indexing
c3=df2.iloc[1:3,2]
c4=df2.iloc[1:3,0:2]
c5=df2.iloc[1:3,:]
c6=df2.iloc[[0,2,3],[1,2]]
TF=[True,False,False,True,True]
c7=df2.iloc[TF,[1]]
```

执行结果如图3-18所示。

# 3.3.4数据框切片



df2 - DataFrame

| Index | a | b | c | d |
|---|---|---|---|---|
| 0 | 2 | k1 | 4 | 7 |
| 1 | 2 | k1 | 6 | 9 |
| 2 | 0 | k1 | 5 | 0 |
| 3 | 5 | 0 | 0 | 9 |
| 4 | 6 | k1 | 6 | 8 |

c3 - Series

| Index | c |
|---|---|
| 1 | 6 |
| 2 | 5 |

c4 - DataFrame

| Index | a | b |
|---|---|---|
| 1 | 2 | k1 |
| 2 | 0 | k1 |

c6 - DataFrame

| Index | b | c |
|---|---|---|
| 0 | k1 | 4 |
| 2 | k1 | 5 |
| 3 | 0 | 0 |

c7 - DataFrame

| Index | b |
|---|---|
| 0 | k1 |
| 3 | 0 |
| 4 | k1 |

图3-18

**2. 利用数据框中的loc属性进行切片**

数据框中的loc属性则主要是基于列标签进行索引，即对列值进行筛选实现行定位，再通过指定列，从而实现数据切片操作。如果取所有列，可以用冒号来表示。切片操作获得的数据还可以筛选前N行。示例代码如下：

```
# loc for label based indexing
c8=df2.loc[df2['b'] == 'kl',:];
c9=df2.loc[df2['b'] == 'kl',:].head(3);
c10=df2.loc[df2['b'] == 'kl',['a','c']].head(3);
c11=df2.loc[df2['b'] == 'kl',['a','c']];
```

执行结果如图3-19所示。

# 3.3.4数据框切片



df2 - DataFrame

| Index | a | b | c | d |
|---|---|---|---|---|
| 0 | 2 | k1 | 4 | 7 |
| 1 | 2 | k1 | 6 | 9 |
| 2 | 0 | k1 | 5 | 0 |
| 3 | 5 | 0 | 0 | 9 |
| 4 | 6 | k1 | 6 | 8 |

c10 - DataFrame

| Index | a | c |
|---|---|---|
| 0 | 2 | 4 |
| 1 | 2 | 6 |
| 2 | 0 | 5 |

c9 - DataFrame

| Index | a | b | c | d |
|---|---|---|---|---|
| 0 | 2 | k1 | 4 | 7 |
| 1 | 2 | k1 | 6 | 9 |
| 2 | 0 | k1 | 5 | 0 |

图3-19

# 3.4外部文件读取



在实际数据挖掘分析中，其业务数据大多存储在外部文件中，比如Excel、TXT等。因此，需要将外部文件读取到Python中进行挖掘分析。Pandas包中提供了非常丰富的函数来读取各种类型的外部数据文件，下面主要介绍Excel和TxT外部文件的读取。

图3-19

# 3.4.1Excel文件读取



通过read_excel()函数读取Excel文件数据，可以读取指定的工作簿（sheet），也可以设置读取有无表头的数据表。示例代码如下：

```
path='一、车次上车人数统计表.xlsx';
data=pd.read_excel(path);
```

执行结果如图3-20所示。

图3-20

读取Sheet2里的数据，示例代码如下：

```
data=pd.read_excel(path,'Sheet2')  #读取sheet里面的数据
```

执行结果图3-21所示。

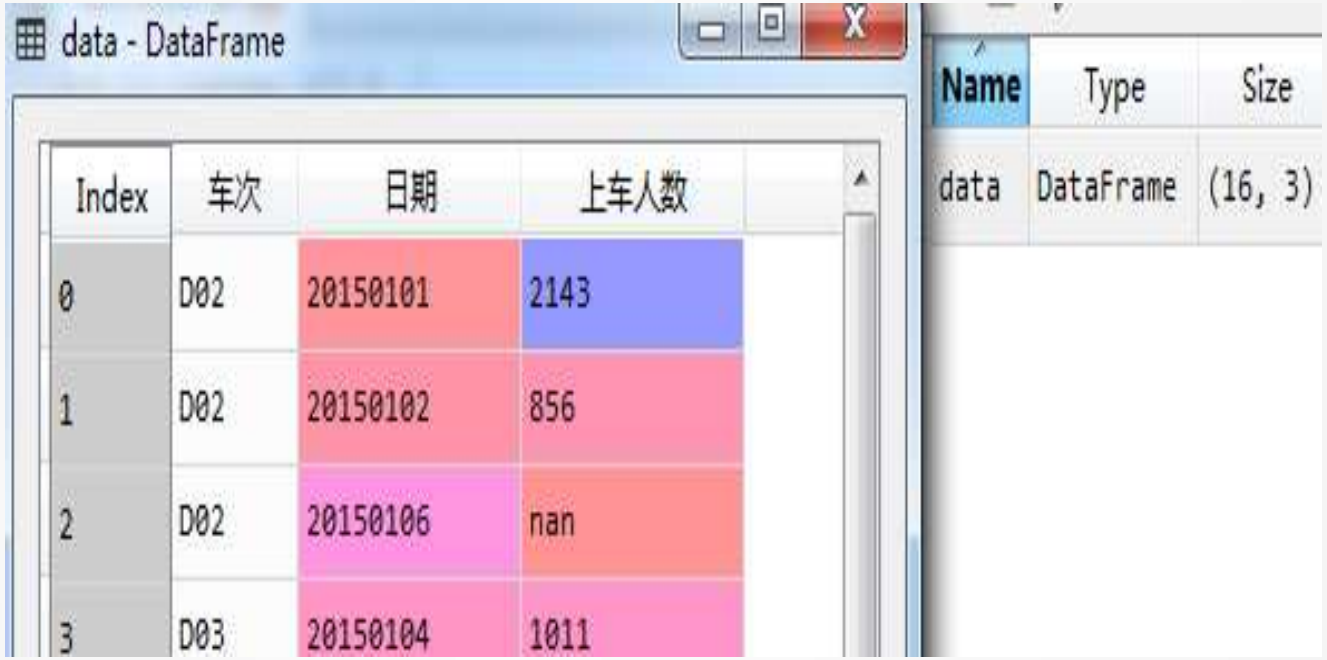



图3-21

# 3.4.1Excel文件读取



有时候数据表中没有设置字段，即无表头，读取格式示例代码如下：

```
dta=pd.read_excel('dta.xlsx',header=None)  #无表头
```

执行结果如图3-22所示。

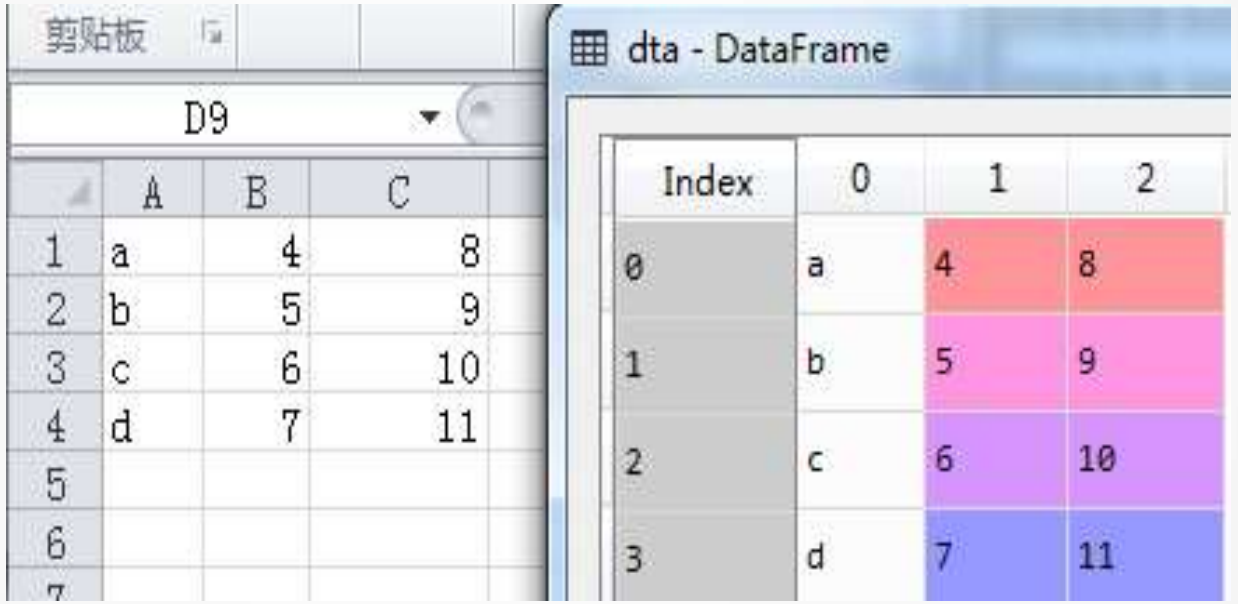



图3-22

# 3.4.2TXT文件读取



通过read_table()函数可以读取TXT文本数据。需要注意的是，TXT文本数据列之间会存在特殊字符作为分隔，常见的有Tab键、空格和逗号。同时还需注意有些文本数据文件是没有设置表头的。示例代码如下：

```
import pandas as pd
dta1=pd.read_table('txt1.txt',header=None)  #分隔默认为Tab键，设置无表头。
```

执行结果如图3-23所示。

# 3.4.2TXT文件读取

图3-23

```
dta2=pd.read_table('txt2.txt',sep='\s+')
#分隔为空格，带表头
```

执行结果如图3-24所示。

图3-24

# 3.4.2TXT文件读取



```
dta3=pd.read_table('txt3.txt',sep=',',header=None)
#分隔为逗号，设置无表头
```

执行结果如图3-25所示。

图3-25

# 3.5滚动计算函数



常用的滚动计算函数有滚动求和rolling_sum()、求平均值rolling_mean()、求最大值rolling_max()、求最小值rolling_min()等。滚动计算函数在金融数据处理中应用非常广泛，比如移动平均价、移动平均量等计算。下面我们对这几个函数进行详细介绍。

滚动求平均值函数的调用形式为：rolling_mean(P,N)，其中P为待求的数据列，N为滚动计算的长度。这里P可以是Numpy数组或者序列数据结构，但是不能是列表或者元组，示例代码如下：

```
import pandas as pd
import numpy as np
L=[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]  #列表
T=(1,2,3,4,5,6,7,8,9,10,11,12,13,14,15)  #元组
A=np.array(L)                            #将列表L转换为数组，赋给变量A
S=pd.Series(L)                           #将列表L转换为序列，赋给变量S
#avg_L=pd.rolling_mean(L,10)   #报错
#avg_T=pd.rolling_mean(T,10)   #报错
avg_S=pd.rolling_mean(S,10)
avg_A=pd.rolling_mean(A,10)
```

执行结果如图3-26所示。

avg_A - NumPy array

| | 0 |
|---|---|
| 0 | nan |
| 1 | nan |
| 2 | nan |
| 3 | nan |
| 4 | nan |
| 5 | nan |
| 6 | nan |
| 7 | nan |
| 8 | nan |
| 9 | 5.5 |
| 10 | 6.5 |
| 11 | 7.5 |
| 12 | 8.5 |
| 13 | 9.5 |
| 14 | 10.5 |

avg_S - Series

| Index | 0 |
|---|---|
| 0 | nan |
| 1 | nan |
| 2 | nan |
| 3 | nan |
| 4 | nan |
| 5 | nan |
| 6 | nan |
| 7 | nan |
| 8 | nan |
| 9 | 5.5 |
| 10 | 6.5 |
| 11 | 7.5 |
| 12 | 8.5 |
| 13 | 9.5 |
| 14 | 10.5 |

图3-26

# 3.5滚动计算函数



其中输入的数据结构为Numpy数组，其返回结果也为Numpy数组；如果输入的数据结构为序列形式，其返回结果也为序列。从返回的结果可以看出，不足滚动计算周期的数据，返回结果均采用nan（空值）来表示，在应用的过程中需要注意把这些数据清洗掉。

同理还有滚动求和函数rolling_sum(P,N)、滚动求最小值函数rolling_min(P,N)和滚动求最大值函数rolling_max(P,N)。示例代码如下：

```
sum_S=pd.rolling_sum(S,10)
sum_A=pd.rolling_sum(A,10)
Min_S=pd.rolling_min(S,10)
Min_A=pd.rolling_min(A,10)
Max_S=pd.rolling_max(S,10)
Max_A=pd.rolling_max(A,10)
```

图3-26

# 本章小结



本章介绍了Python数据处理与分析中最重要包：Pandas。包括Pandas包的导入及使用方法，Pandas库中两个非常重要的数据结构序列（Series）和数据框（DataFrame），以及相关的数据访问、切片及计算。值得注意的是，读者需要掌握数据框、序列和Numpy数组之间的关系。从数据框中取出一列，它变为序列，取序列中的values属性得到序列的值，它其实是Numpy数组。从数据框中切片出来多个数据列，它仍然是数据框。取数据框中的values属性得到数据框中的元素值，它是一个Numpy数组。如果数据框中的元素是纯数值类型，可以通过函数as_matrix()直接转换为Numpy数组，这样在计算和使用起来显得更加方便。同时我们还应该注意数据框与外部数据文件的读写，特别是Excel文件，它为数据报表的产生提供了极大的便利。在程序编写过程我们还应该注意不同数据类型之间的转换，比如通过字典可以转换为数据框，其中字典的键转化为数据框中的列名，字典的值转化为数据框中的元素值，而字典的值可以是列表或者数组。这样就实现了列表、字典、数组、序列、数据框等各种数据类型和数据结构之间的相互转化，从而完成各种计算任务。事实上，不同数据结构之间的相互转化也是一种非常重要的编程技能和应用技巧，后续在案例篇中会有具体应用，请读者们到时注意领会。在本章的最后也介绍了Pandas包中的外部文件读取方法和利用Pandas包中的函数完成计算任务。Pandas包的内容非常丰富，本章只是介绍其基本部分，更多的内容请查找相关文献或者借助网络资源进行学习。

第一题

创建一个Python脚本，命名为test1.py，完成以下功能：

读取以下4位同学的成绩并用一个数据框变量pd来保存，其中成绩保存在一个TXT文件中，如下图所示：

成绩单 - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
姓名，科目，成绩
小红，语文，100
小红，英语，90
小红，数学，75
张明，语文，80
张明，英语，76
张明，数学，88
小江，语文，79
小江，数学，120
小江，英语，80
小李，英语，87
小李，语文，99
小李，数学，76
```

对pd进行切片操作，分别获得小红、张明、小江、小李各科成绩，它们是4个数据框变量，分别记为pd1、pd2、pd3、pd4。

第二题

利用数据框中自身的聚合计算方法，计算并获得每个同学各科成绩的平均分，记为M1、M2、M3、M4。

创建一个Python脚本，命名为test2.py，完成以下功能：

读取以下Excel表格数据并用一个数据框变量df来保存，数据内容如下表所示：

| 股票代码 | 交易日期 | 收盘价 | 交易量 |
|---|---|---|---|
| 600000 | 2017-01-03 | 16.3 | 16237125 |
| 600000 | 2017-01-04 | 16.33 | 29658734 |
| 600000 | 2017-01-05 | 16.3 | 26437646 |
| 600000 | 2017-01-06 | 16.18 | 17195598 |
| 600000 | 2017-01-09 | 16.2 | 14908745 |
| 600000 | 2017-01-10 | 16.19 | 7996756 |
| 600000 | 2017-01-11 | 16.16 | 9193332 |
| 600000 | 2017-01-12 | 16.12 | 8296150 |
| 600000 | 2017-01-13 | 16.27 | 19034143 |
| 600000 | 2017-01-16 | 16.56 | 53304724 |
| 600000 | 2017-01-17 | 16.4 | 12555292 |
| 600000 | 2017-01-18 | 16.48 | 11478663 |
| 600000 | 2017-01-19 | 16.54 | 12180687 |
| 600000 | 2017-01-20 | 16.6 | 14288268 |

对df第3、4列进行切片，切片后为一个新的数据框记为df1，并对df1利用自身的方法转换为Numpy数组Nt。

基于df第2列，构造一个逻辑数组TF，即满足交易日期小于等于2017-01-16且大于等于2017-01-05为真，否则为假。

以TF为索引，取Nt中的第2列交易量数据并求和，记为S。